

Przeptyw danych - faza A

Sciezki: dodawanie zgloszen i wysylka batcha (Wniosku BNC)

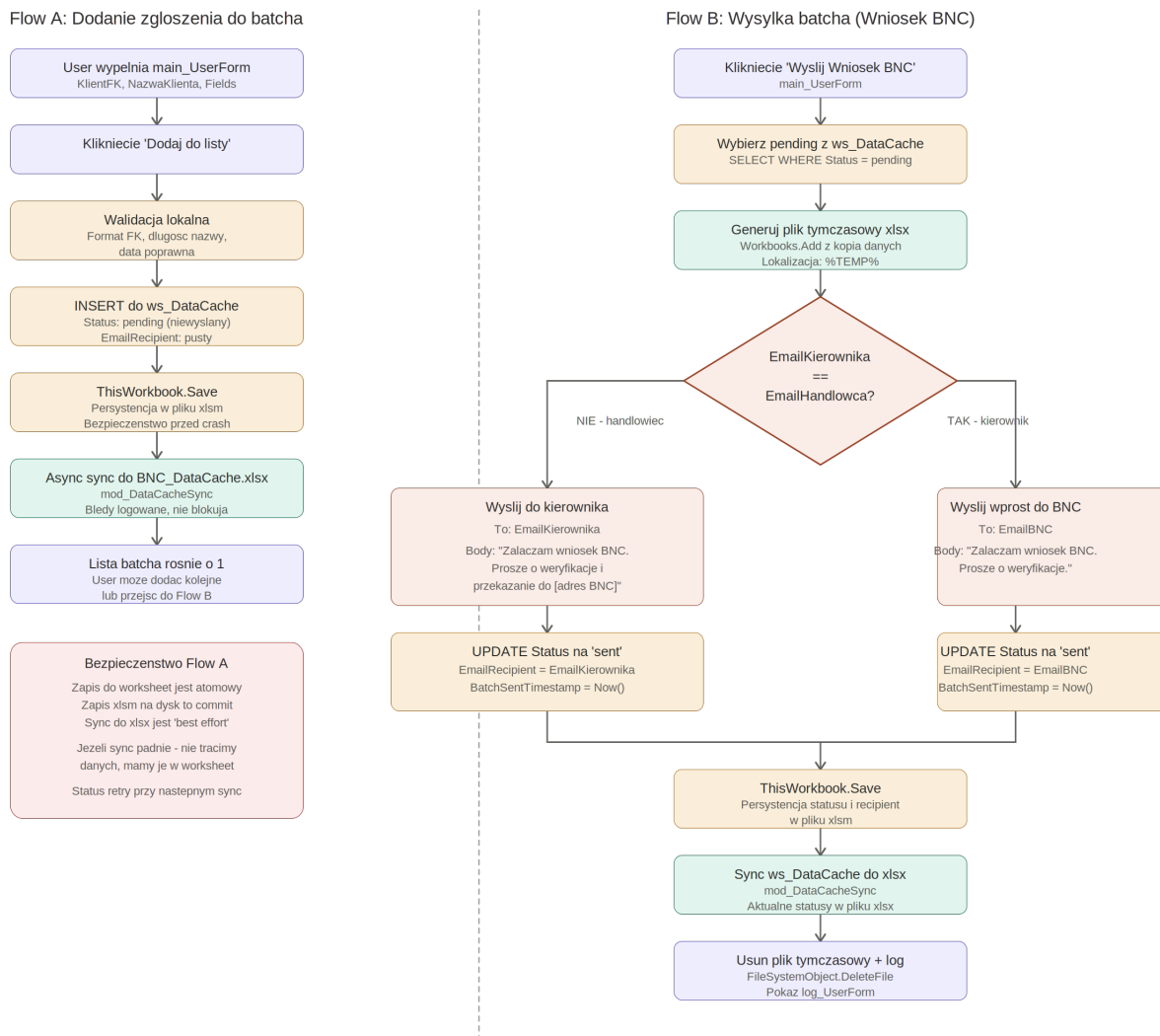


Diagram: Przeptyw danych z decision diamond

Cel diagramu

Diagram pokazuje dwa podstawowe przeptyw danych w aplikacji. Flow A to dodawanie pojedynczego zgłoszenia do batcha. Flow B to wysyłka batcha pending zgłoszen jako Wniosek BNC. Wewnątrz Flow B znajduje się decision diamond rozdzielający logikę wysyłki w zależności od roli usera.

Flow A - dodawanie zgłoszenia do batcha

User wypełnia main_UserForm wprowadzając KlientFK, NazwaKlienta i pozostałe pola. Po kliknięciu Dodaj do listy aplikacja waliduje lokalnie format pol. Po pomyslniej walidacji INSERT do ws_DataCache z polami Status=pending i EmailRecipient=pusty. Następnie ThisWorkbook.Save persystuje zmianę w pliku xlsx na dysku. Dopiero po tym mod_DataCacheSync próbuje skopiować stan do BNC_DataCache.xlsx - jeżeli się nie udaje, błąd jest logowany ale nie blokuje usera.

Flow B - wysyłka Wniosku BNC z decision diamond

User klika Wyslij Wniosek BNC. Aplikacja wybiera z ws_DataCache wszystkie wiersze ze statusem pending. Generuje plik tymczasowy xlsx w %TEMP%. Następnie następuje decision diamond - sprawdzenie czy user jest kierownikiem.

Logika kierownik vs handlowiec - convention over configuration

Aplikacja rozpoznaje rolę usera bez dodatkowych pól konfiguracyjnych - wykorzystuje porównanie EmailKierownika z EmailHandlowca. Jeżeli są równe, oznacza to że user wpisał sam siebie jako swojego kierownika - czyli jest kierownikiem. To jest wzorzec convention over configuration. Brak nowego pola, brak nowej tabeli, jedna reguła w kodzie. Konkretnie:

```
If ws_UserCache.EmailKierownika = ws_UserCache.EmailHandlowca Then
    REM User jest kierownikiem - wysyłamy wprost do BNC
    To = ws_UserCache.EmailBNC
    Body = "Zalaczam wniosek BNC. Proszę o weryfikację."
    EmailRecipient = ws_UserCache.EmailBNC
Else
    REM User jest handlowcem - wysyłamy do kierownika
    To = ws_UserCache.EmailKierownika
    Body = "Zalaczam wniosek BNC. Proszę o weryfikację i przekazanie do " _
        & ws_UserCache.EmailBNC
    EmailRecipient = ws_UserCache.EmailKierownika
End If
```

Status sent w kontekście audytu

Po pomyslniej wysyłce UPDATE w ws_DataCache - wszystkie wysłane wiersze dostają Status=sent oraz BatchSentTimestamp. Dodatkowo aplikacja zapisuje kolumnę EmailRecipient z rzeczywistym adresatem. To jest audit trail - przy reklamacji BNC nie dostało handlowiec może pokazać wysłałem do mojego kierownika dnia X, sprawdź u niego. Z perspektywy aplikacji wysłane do kierownika to status sent - ale dzięki polu EmailRecipient mamy pełną informację, KOMU faktycznie wysłaliśmy.

Mechanizm pliku tymczasowego w %TEMP%

Czym jest %TEMP% i jak Windows go zarządza

%TEMP% to zmienna środowiskowa Windows wskazująca na folder tymczasowych plików konkretnego usera. Standardowa lokalizacja:

```
C:\Users\[login_usera]\AppData\Local\Temp\
```

Każdy user Windows ma swój własny folder TEMP. Charakterystyki istotne dla aplikacji:

- Zawsze writable dla zalogowanego usera - to jego własny obszar, bez UAC
- Auto-czyszczony przez Storage Sense w Windows 10/11 (po kilkunastu dniach)
- Wykluczony z synchronizacji OneDrive domyślnie
- Wykluczony z backupów w większości polityk korporacyjnych
- Niewidoczny dla usera w Eksploratorze (bo AppData jest hidden)

Czy Excel potrzebuje dodatkowych pozwoleń?

Krótką odpowiedź: NIE. Folder %TEMP% jest własnością usera, każda aplikacja działająca w jego kontekście ma do niego dostęp bez dodatkowych uprawnień. VBA + Excel COM do operacji na plikach (Workbooks.Add, SaveAs, Close, FileSystemObject.DeleteFile) to standardowy workflow, który działa out of the box.

Cykl zycia pliku tymczasowego krok po kroku

```
Public Sub SendBatchAsAttachment()  
    Dim tempFolderPath As String  
    Dim tempFileName As String  
    Dim tempFilePath As String  
  
    REM 1. Sciezka do folderu TEMP  
    tempFolderPath = Environ("TEMP")  
  
    REM 2. Unikalna nazwa pliku (uniknij kolizji)  
    tempFileName = "BNC_Wniosek_" _  
        & Format(Now(), "yyyymmdd_hhmmss") _  
        & ".xlsx"  
    tempFilePath = tempFolderPath & "\\\" & tempFileName  
  
    REM 3. Stworz plik xlsx z aktualnym batchem  
    Dim wb As Workbook  
    Set wb = Workbooks.Add  
    REM ... wypelnij dane z ws_DataCache WHERE Status='pending' ...  
    wb.SaveAs Filename:=tempFilePath, FileFormat:=xlOpenXMLWorkbook  
    wb.Close SaveChanges:=False  
  
    REM 4. Zalacz do maila i wyslij (decision diamond - kierownik vs BNC)  
    REM ... mod_MailSender.SendWithAttachment(tempFilePath) ...  
  
    REM 5. Posprzataj  
    Dim fso As Object  
    Set fso = CreateObject("Scripting.FileSystemObject")  
    If fso.FileExists(tempFilePath) Then  
        fso.DeleteFile tempFilePath  
    End If  
End Sub
```

Co sie dzieje gdy mail sie nie wysle

W kroku 4 wystapi blad. Plik tymczasowy zostaje na dysku (krok 5 nie zostal wykonany). Aplikacja powinna obsluzyc blad: pokazac komunikat userowi, zaproponowac ponowna probe, oraz wyczyszcic plik tymczasowy w error handler. Klasyczny VBA error handling uzywa wzorca On Error GoTo Cleanup z sekcja Cleanup zawierajaca DeleteFile w finally.

Dlaczego nie zostawiamy pliku jako historie

Bo historia zgloszen jest juz w BNC_DataCache.xlsx - to jest prawdziwy backup. Plik tymczasowy w %TEMP% to transient artifact - sluzzy tylko do jednej operacji (wysluki jako zalacznik) i jest natychmiast niepotrzebny. Pozostawienie go zaśmiecaloby dysk i mogloby konfundowac usera (po co mam plik BNC_Wniosek_20260505_142311.xlsx?).